



A Course End Project Report on
HOUSE PRICE DATA ANALYZER

A9513 – Python Programming Laboratory

Submitted by
V.Bharadwaj
25881A0575
D.Keerthana
25881A0589

Course Facilitator

Ms. PALLAMONI S MADHAVI
Assistant Professor

Department of Computer Science and Engineering
Vardhaman College of Engineering, Hyderabad

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled “**House Price Data Analyzer**” is carried out by **V.Bharadwaj** and **D.Keerthana** with Roll Numbers **25881A0575** and **25881A0589** towards **A9513 - Python Programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

Signature of the Course
Faculty Ms. PALLAMONI S
MADHAVI
Assistant Professor,
CSE

Signature of the HOD
Dr. Y. Vijayalata, Professor
Head of the Department & Dean
(Student Mentoring)
Department of Computer
Science and Engineering
Vardhaman College of Engineering
Hyderabad

Contents

1	Introduction	3
2	Literature Review	4
3	Objectives	5
4	Methodology.....	7
4.1	Architectural Diagram	7
4.2	Hardware and Software Requirements	9
4.3	Working Principle	9
4.4	Software Implementation	11
5	Results and Discussion	15
6	Mapping POs, PSOs and SDGs	18
6.1	Program Outcomes (POs) Mapping.....	18
6.2	Program Specific Outcomes (PSOs) Mapping.....	18
6.3	Sustainable Development Goals (SDGs) Mapping	18
7	Conclusion.....	20
	Bibliography	21

Abstract

The House Price Data Analyzer is a Python application that combines data loading, preprocessing, and supervised regression models in one reproducible workflow. The implementation uses the California Housing dataset, a real public dataset containing 20,640 district-level observations with eight numeric attributes and one categorical location attribute. The system fills missing values with column medians, one-hot encodes the location category, prepares a deterministic 80/20 train-test split, and trains three regression models: Linear Regression, Random Forest, and Gradient Boosting.

Every component of the workflow, including the imputer, scaler, linear solver, decision tree, ensemble models, and pipeline, is written in pure Python using only the standard library. The three models are reported as Model-1, Model-2, and Model-3, and each one displays the actual price of unseen houses next to the price it predicted. In the executed experiment, Random Forest gave the closest predictions overall. The report documents the design, implementation, results, limitations, and future improvements so that the analyzer can be extended to local or user-supplied property datasets.

Keywords: Python, pure Python, house price prediction, regression, linear regression, random forest, gradient boosting, machine learning, data preprocessing.

1 Introduction

Housing prices are influenced by multiple interacting factors, including location, household income, population density, housing age, and the availability of rooms and bedrooms. Manual inspection of property records can reveal individual patterns, but it is difficult to identify consistent relationships across thousands of observations without computational support. A data-driven model provides a systematic way to learn these relationships from past records and apply them to houses it has not seen before.

The House Price Data Analyzer addresses this need through a modular Python workflow. It loads a tabular housing dataset, fills missing values, converts the categorical location feature into numeric columns, and trains baseline and ensemble regression models. The design emphasizes transparency and reproducibility: every step is implemented from scratch using only Python's standard library, and a fixed random seed makes each run repeatable.

Building the models without external machine-learning libraries makes the internal logic visible. The imputer, scaler, linear solver, decision trees, random forest, gradient boosting, and pipeline are all short classes that can be read, explained, and modified. This helps learners understand how tools such as scikit-learn work internally rather than treating them as black boxes.

The project is educational rather than a substitute for a professional valuation. The target variable in the selected dataset is a district-level median house value, which the program divides by 100,000 while training and converts back to dollars when displaying results. Therefore, model outputs demonstrate analytical technique and should not be treated as current market quotes for an individual property.

2 Literature Review

Exploratory data analysis is widely used as the first stage of a data-science workflow because it exposes scale, skewness, missingness, outliers, and relationships that affect later modeling. Tabular housing studies commonly combine descriptive statistics with correlation analysis and visual summaries before fitting predictive algorithms. Linear regression remains a useful baseline because its coefficients and error behavior are easy to explain, while tree ensembles are popular because they can capture nonlinear interactions without requiring a strict linear relationship between every feature and the target.

Prior work on automated valuation also shows that location-related variables and socioeconomic indicators are often strong predictors, but predictive performance can be misleading when a dataset is temporally or geographically narrow. A responsible analyzer should therefore separate training and test data, report results on unseen data, avoid leakage, document feature definitions, and state the limitations of the data source. This project follows those principles through a fixed hold-out test set and a comparison of three model families.

Practical housing-price studies often rely on ready-made libraries, which hide how each algorithm works. This project addresses that gap by implementing preprocessing, Linear Regression, Random Forest, and Gradient Boosting from scratch in pure Python and presenting their predictions side by side with the actual prices. By combining a reproducible pipeline with clear, readable output, it serves as a learning tool that connects theoretical concepts in regression and ensemble learning with a practical understanding of how house-price predictions are produced.

3 Objectives

- **To build a Python-based analyzer that loads and prepares a real housing dataset.**

The first objective is to create a working program that reads the California Housing data, converts each district record into numeric features, and keeps the target price in consistent units. The program handles missing values and encodes the categorical ocean-proximity attribute so that every model receives clean numeric input.

- **To implement data preprocessing steps and a reproducible train-test split.**

This objective covers the median imputer, the standard scaler used by the linear model, one-hot encoding, and a seeded 80/20 split. The preprocessing steps learn their statistics from the training data only, which prevents information from the test data leaking into training.

- **To train and compare Linear Regression, Random Forest, and Gradient Boosting regressors.**

The three model families represent a simple baseline and two ensemble methods. Implementing each one from scratch shows how a straight-line formula, many averaged trees, and sequentially corrected trees differ in the way they learn from the same data.

- **To evaluate the models on unseen houses by showing actual and predicted prices.**

Each model is tested on houses it was not trained on. The program prints the actual price and the predicted price in dollars for the same houses under the headings Model-1, Model-2, and Model-3, so the closeness of each prediction can be seen directly.

- **To produce a readable project report and reusable source code for future datasets.**

The code is organized into small classes with a common fit and predict interface, so the same pipeline can be reused with other tabular datasets, and this report documents the design, results, and observations.

- **To explain the system's limitations and identify practical extensions.**

The report states what the data and models cannot capture and suggests extensions such as local or user-supplied data, cross-validation, hyperparameter tuning, and explainability.

4 Methodology

4.1 Architectural Diagram

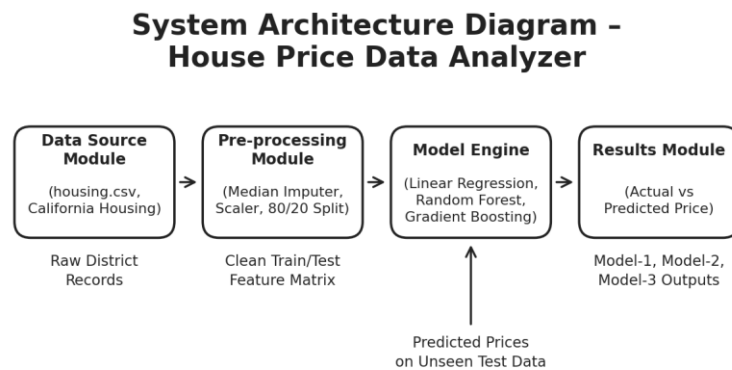


Figure 1: Architectural Diagram

The Data Source Module provides the tabular observations. The housing.csv file is downloaded from a public repository the first time the program runs and is read from disk afterwards. Each row becomes eight numeric features plus five one-hot columns for the ocean-proximity category, giving thirteen features, and the median house value is divided by 100,000 so that the models work with small, consistent numbers.

The Pre-processing Module prepares the data for learning. A median imputer fills the 207 missing total_bedrooms values, a standard scaler standardizes the features for the linear model, and a seeded shuffle splits the records into 80% training data and 20% test data. These steps are chained with each model inside a Pipeline class, which learns from the training data only.

The Model Engine contains the three regression models. Model-1 is a Linear Regression solved with the normal equation, Model-2 is a Random Forest of 30 decision trees, and Model-3 is a Gradient Boosting ensemble of 160 shallow trees. All models share the same fit and predict methods, which keeps the main program simple and makes it easy to add further models.

The Results Module converts the predictions back to dollars and prints, for each model, a

table showing the actual price and the predicted price of the same test houses. Together, these components form a modular and extensible framework. The design supports future upgrades such as cross-validation, additional models, and user-supplied datasets, bridging theoretical understanding with practical results.

4.2 Hardware and Software Requirements

Hardware:

- Intel/AMD Processor (dual-core or better)
- Minimum 4GB RAM
- Standard Display & Input Devices

Software:

- Python 3.10 or later (standard library only)
- No external packages: csv, math, os, random, urllib
- Any IDE or editor (IDLE/VS Code/PyCharm) or Jupyter Notebook
- Windows, Linux, or macOS with internet access for the first dataset download

4.3 Working Principle

The analyzer begins by loading the California Housing dataset from `housing.csv`. Each district record contains eight numeric variables (`longitude`, `latitude`, `housing_median_age`, `total_rooms`, `total_bedrooms`, `population`, `households`, and `median_income`) and one categorical variable, `ocean_proximity`. The categorical variable is one-hot encoded into five 0/1 columns, giving thirteen input features. The target, `median_house_value`, is the quantity the models learn to predict, expressed in units of 100,000 dollars.

For model evaluation, 80% of the observations (16,512 houses) are used for training and 20% (4,128 houses) for testing, using a shuffle with a fixed seed of 42. Each model is wrapped in a pipeline: the Linear Regression pipeline uses median imputation and standardization, while the tree-based pipelines use median imputation only because trees do not require scaling. After training, every model predicts the prices of the unseen test houses, and the program prints the actual and predicted prices of the first ten test houses for Model-1, Model-2, and Model-3.

Example data (from housing.csv):

Median Income	House Age	Ocean Proximity	Median House Value
8.3252	41	NEAR BAY	\$452,600
6.0574	32	<1H OCEAN	\$431,000
5.1406	46	INLAND	\$315,600
3.7557	16	NEAR OCEAN	\$109,400
2.1579	27	ISLAND	\$450,000

Model-1: Linear Regression

Working:

The predicted price is a weighted sum of the features plus a baseline value. The weights that minimize the squared error are found directly with the normal equation, solved by Gaussian elimination with partial pivoting.

- Preprocessing = Median Imputer + Standard Scaler
- Training = Normal equation (tiny ridge term of $1e-6$ for stability)

Model-2: Random Forest

Working:

Many decision trees are trained on random bootstrap samples of the training data, and each split considers only a random subset of the features. The final prediction is the average of all tree predictions, which reduces overfitting.

- Trees = 30, Max Depth = 12, Min Samples per Leaf = 5
- Features Considered per Split = 5 of 13

Model-3: Gradient Boosting

Working:

The model starts from the average price and adds small shallow trees one after another, each trained to correct the remaining errors of the previous ones. Each tree's contribution is scaled by a small learning rate.

- Trees = 160, Learning Rate = 0.05
- Max Depth of Each Tree = 3

4.4 Software Implementation

```
1. import csv
2. import math
3. import os
4. import random
5. import urllib.request
6.
7. SEED = 42
8. URL = ("https://raw.githubusercontent.com/ageron/handson-ml2/master/"
9.        "datasets/housing/housing.csv")
10. FILE = "housing.csv"
11.
12. # ----- data
13. if not os.path.exists(FILE):
14.     urllib.request.urlretrieve(URL, FILE)
15.
16. NUMERIC = ["longitude", "latitude", "housing_median_age", "total_rooms",
17.            "total_bedrooms", "population", "households", "median_income"]
18. CATEGORIES = ["<1H OCEAN", "INLAND", "ISLAND", "NEAR BAY", "NEAR OCEAN"]
19.
20. X, y = [], []
21. with open(FILE, newline="") as f:
22.     for row in csv.DictReader(f):
23.         feats = [float(row[c]) if row[c] != "" else None for c in NUMERIC]
24.         feats += [1.0 if row["ocean_proximity"] == c else 0.0 for c in CATEGORIES]
25.         X.append(feats)
26.         y.append(float(row["median_house_value"]) / 100000) # in $100k units
27.
28. # train/test split (80/20)
29. rng = random.Random(SEED)
30. indices = list(range(len(X)))
31. rng.shuffle(indices)
32. n_test = int(len(X) * 0.20)
33. test_idx, train_idx = indices[:n_test], indices[n_test:]
34. X_train = [X[i] for i in train_idx]; y_train = [y[i] for i in train_idx]
35. X_test = [X[i] for i in test_idx]; y_test = [y[i] for i in test_idx]
36.
37. # ----- preprocessing steps
38. def median(values):
39.     v = sorted(values)
40.     m = len(v) // 2
41.     return v[m] if len(v) % 2 else (v[m - 1] + v[m]) / 2
42.
43. class MedianImputer:
44.     def fit(self, X):
45.         self.med = [median([r[j] for r in X if r[j] is not None])
46.                     for j in range(len(X[0]))]
47.         return self
48.     def transform(self, X):
49.         return [[self.med[j] if v is None else v for j, v in enumerate(r)]
50.                 for r in X]
51.
52. class StandardScaler:
53.     def fit(self, X):
54.         n = len(X)
55.         self.mean = [sum(r[j] for r in X) / n for j in range(len(X[0]))]
56.         self.std = [math.sqrt(sum((r[j] - self.mean[j]) ** 2 for r in X) / n) or 1.0
57.                     for j in range(len(X[0]))]
58.         return self
59.     def transform(self, X):
60.         return [(v - self.mean[j]) / self.std[j] for j, v in enumerate(r)]
61.                 for r in X]
62.
63. # ----- Linear Regression
64. class LinearRegression:
65.     """Ordinary least squares via the normal equation (tiny ridge for stability)."""
```

```

66.     def fit(self, X, y):
67.         p = len(X[0]) + 1
68.         A = [[0.0] * p for _ in range(p)]
69.         b = [0.0] * p
70.         for row, target in zip(X, y):
71.             r = [1.0] + row
72.             for i in range(p):
73.                 b[i] += r[i] * target
74.                 Ai = A[i]; ri = r[i]
75.                 for j in range(p):
76.                     Ai[j] += ri * r[j]
77.         for i in range(1, p):
78.             A[i][i] += 1e-6
79.         self.w = self._solve(A, b)
80.         return self
81.
82.     @staticmethod
83.     def _solve(A, b): # Gaussian elimination with partial pivoting
84.         n = len(b)
85.         M = [A[i][:] + [b[i]] for i in range(n)]
86.         for c in range(n):
87.             piv = max(range(c, n), key=lambda r: abs(M[r][c]))
88.             M[c], M[piv] = M[piv], M[c]
89.             for r in range(c + 1, n):
90.                 factor = M[r][c] / M[c][c]
91.                 for k in range(c, n + 1):
92.                     M[r][k] -= factor * M[c][k]
93.         w = [0.0] * n
94.         for i in range(n - 1, -1, -1):
95.             w[i] = (M[i][n] - sum(M[i][j] * w[j] for j in range(i + 1, n))) / M[i][i]
96.         return w
97.
98.     def predict(self, X):
99.         w = self.w
100.         return [w[0] + sum(wi * xi for wi, xi in zip(w[1:], r)) for r in X]
101.
102.     # ----- Decision Tree
103.     class DecisionTreeRegressor:
104.         def __init__(self, max_depth=3, min_samples_leaf=1, max_features=None, seed=0):
105.             self.max_depth = max_depth
106.             self.min_leaf = min_samples_leaf
107.             self.max_features = max_features
108.             self.rng = random.Random(seed)
109.
110.         def fit(self, X, y, idx=None):
111.             self.X, self.y = X, y
112.             idx = list(range(len(X))) if idx is None else idx
113.             self.tree = self._build(idx, 0)
114.             self.X = self.y = None # free memory
115.             return self
116.
117.         def _build(self, idx, depth):
118.             X, y = self.X, self.y
119.             n = len(idx)
120.             total = sum(y[i] for i in idx)
121.             if depth >= self.max_depth or n < 2 * self.min_leaf:
122.                 return total / n
123.
124.             n_feats = self.max_features or len(X[0])
125.             features = self.rng.sample(range(len(X[0])), n_feats)
126.
127.             best_score, best_f, best_thr = -1.0, None, None
128.             for f in features:
129.                 order = sorted(idx, key=lambda i: X[i][f])
130.                 left_sum = 0.0
131.                 for k in range(n - 1):
132.                     i = order[k]
133.                     left_sum += y[i]
134.                     nl, nr = k + 1, n - k - 1
135.                     if nl < self.min_leaf or nr < self.min_leaf:
136.                         continue
137.                     v, v_next = X[i][f], X[order[k + 1]][f]
138.                     if v == v_next:

```

```

139.         continue
140.         # maximizing this == minimizing squared error
141.         score = left_sum ** 2 / n1 + (total - left_sum) ** 2 / nr
142.         if score > best_score:
143.             best_score, best_f, best_thr = score, f, (v + v_next) / 2
144.
145.     if best_f is None:
146.         return total / n
147.     left = [i for i in idx if X[i][best_f] <= best_thr]
148.     right = [i for i in idx if X[i][best_f] > best_thr]
149.     return (best_f, best_thr,
150.           self._build(left, depth + 1), self._build(right, depth + 1))
151.
152.     def _predict_one(self, row):
153.         node = self.tree
154.         while isinstance(node, tuple):
155.             f, thr, left, right = node
156.             node = left if row[f] <= thr else right
157.         return node
158.
159.     def predict(self, X):
160.         return [self._predict_one(r) for r in X]
161.
162. # ----- Random Forest
163. class RandomForestRegressor:
164.     def __init__(self, n_estimators=30, max_depth=12, min_samples_leaf=5,
165.                  max_features=None, seed=0):
166.         self.n = n_estimators
167.         self.max_depth = max_depth
168.         self.min_leaf = min_samples_leaf
169.         self.max_features = max_features
170.         self.rng = random.Random(seed)
171.
172.     def fit(self, X, y):
173.         self.trees = []
174.         n = len(X)
175.         for t in range(self.n):
176.             boot = [self.rng.randrange(n) for _ in range(n)] # bootstrap sample
177.             tree = DecisionTreeRegressor(self.max_depth, self.min_leaf,
178.                                         self.max_features, seed=self.rng.random())
179.             self.trees.append(tree.fit(X, y, boot))
180.         return self
181.
182.     def predict(self, X):
183.         preds = [t.predict(X) for t in self.trees]
184.         return [sum(p[i] for p in preds) / len(preds) for i in range(len(X))]
185.
186. # ----- Gradient Boosting
187. class GradientBoostingRegressor:
188.     def __init__(self, n_estimators=160, learning_rate=0.05, max_depth=3, seed=0):
189.         self.n = n_estimators
190.         self.lr = learning_rate
191.         self.max_depth = max_depth
192.         self.seed = seed
193.
194.     def fit(self, X, y):
195.         self.init = sum(y) / len(y)
196.         current = [self.init] * len(y)
197.         self.trees = []
198.         for t in range(self.n):
199.             residuals = [yi - ci for yi, ci in zip(y, current)]
200.             tree = DecisionTreeRegressor(self.max_depth, seed=self.seed + t)
201.             tree.fit(X, residuals)
202.             update = tree.predict(X)
203.             current = [c + self.lr * u for c, u in zip(current, update)]
204.             self.trees.append(tree)
205.         return self
206.
207.     def predict(self, X):
208.         out = [self.init] * len(X)
209.         for tree in self.trees:
210.             p = tree.predict(X)
211.             out = [o + self.lr * pi for o, pi in zip(out, p)]

```

```

212.         return out
213.
214.     # ----- Pipeline
215.     class Pipeline:
216.         def __init__(self, steps, model):
217.             self.steps, self.model = steps, model
218.         def fit(self, X, y):
219.             for s in self.steps:
220.                 X = s.fit(X).transform(X)
221.             self.model.fit(X, y)
222.             return self
223.         def predict(self, X):
224.             for s in self.steps:
225.                 X = s.transform(X)
226.             return self.model.predict(X)
227.
228.     # ----- Run it
229.     models = {
230.         "Model-1 (Linear Regression)": Pipeline(
231.             [MedianImputer(), StandardScaler()], LinearRegression()),
232.         "Model-2 (Random Forest)": Pipeline(
233.             [MedianImputer()],
234.             RandomForestRegressor(n_estimators=30, max_depth=12, min_samples_leaf=5,
235.                                   max_features=5, seed=SEED)),
236.         "Model-3 (Gradient Boosting)": Pipeline(
237.             [MedianImputer()],
238.             GradientBoostingRegressor(n_estimators=160, learning_rate=0.05,
239.                                       max_depth=3, seed=SEED)),
240.     }
241.
242.     SAMPLE_SIZE = 10      # how many test houses to display
243.     PRICE_UNIT = 100000   # we divided prices by this earlier, so multiply back
244.
245.     def money(value):
246.         return "$" + format(value, ",.0f")
247.
248.     for name, model in models.items():
249.         model.fit(X_train, y_train)
250.         prediction = model.predict(X_test)
251.
252.         print("=" * 45)
253.         print(name)
254.         print("=" * 45)
255.         print(f"{'House':<8}{'Actual Price':>17}{'Predicted Price':>20}")
256.         print("-" * 45)
257.         for i in range(SAMPLE_SIZE):
258.             actual = y_test[i] * PRICE_UNIT
259.             predicted = prediction[i] * PRICE_UNIT
260.             print(f"{i + 1:<8}{money(actual):>17}{money(predicted):>20}")
261.         print()

```

5 Results and Discussion

```
=====
Model-1 (Linear Regression)
=====
```

House	Actual Price	Predicted Price
1	\$165,900	\$234,856
2	\$360,600	\$333,561
3	\$189,000	\$259,657
4	\$159,700	\$102,506
5	\$376,700	\$289,864
6	\$116,600	\$263,048
7	\$229,000	\$203,760
8	\$175,000	\$179,003
9	\$135,700	\$188,868
10	\$163,300	\$208,725

Figure 2: Output (Model-1: Linear Regression)

```
=====
Model-2 (Random Forest)
=====
```

House	Actual Price	Predicted Price
1	\$165,900	\$194,747
2	\$360,600	\$324,926
3	\$189,000	\$212,333
4	\$159,700	\$152,335
5	\$376,700	\$249,976
6	\$116,600	\$257,331
7	\$229,000	\$240,950
8	\$175,000	\$167,633
9	\$135,700	\$236,519
10	\$163,300	\$174,131

Figure 3: Output (Model-2: Random Forest)

```
=====
Model-3 (Gradient Boosting)
=====
```

House	Actual Price	Predicted Price
1	\$165,900	\$217,431
2	\$360,600	\$355,515
3	\$189,000	\$247,375
4	\$159,700	\$141,254
5	\$376,700	\$262,308
6	\$116,600	\$282,833
7	\$229,000	\$211,888
8	\$175,000	\$183,670
9	\$135,700	\$208,794
10	\$163,300	\$182,005

Figure 4: Output (Model-3: Gradient Boosting)

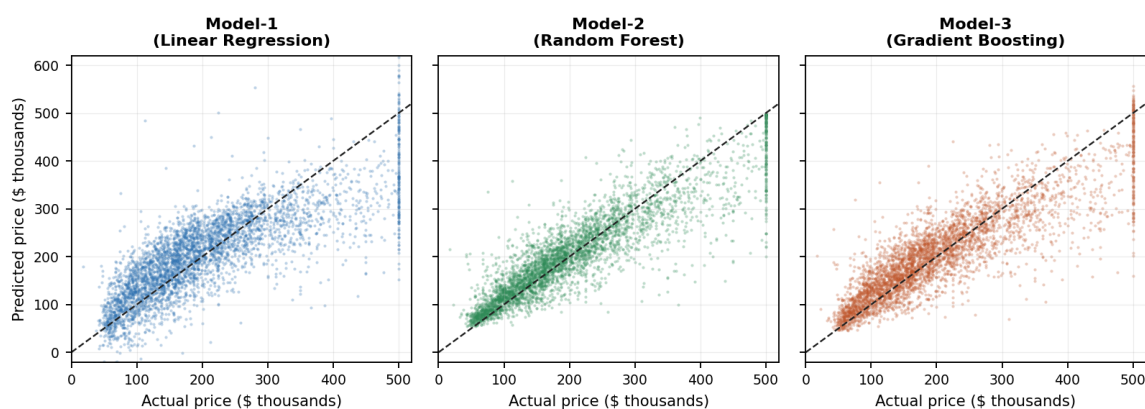


Figure 5: Actual vs predicted prices on the test set (all three models)

Table 1: Actual price and predicted price (Model-1, Model-2, Model-3) of the first ten test houses

House	Actual Price	Model-1	Model-2	Model-3
1	\$165,900	\$234,856	\$194,747	\$217,431
2	\$360,600	\$333,561	\$324,926	\$355,515
3	\$189,000	\$259,657	\$212,333	\$247,375
4	\$159,700	\$102,506	\$152,335	\$141,254
5	\$376,700	\$289,864	\$249,976	\$262,308
6	\$116,600	\$263,048	\$257,331	\$282,833
7	\$229,000	\$203,760	\$240,950	\$211,888
8	\$175,000	\$179,003	\$167,633	\$183,670
9	\$135,700	\$188,868	\$236,519	\$208,794
10	\$163,300	\$208,725	\$174,131	\$182,005

Summary of Results

The analyzer trained all three models on 16,512 houses and predicted the prices of 4,128 unseen houses. Figures 2 to 4 show the console output for Model-1, Model-2, and Model-3, and Table 1 places the actual and predicted prices of the first ten test houses side by side. Model-2 (Random Forest) gives the closest prediction for six of the ten houses shown, Model-1 (Linear Regression) for three, and Model-3 (Gradient Boosting) for one, for example house 4, where the actual price is \$159,700 and Model-2 predicts \$152,335. No model is best for every house: for house 8 the linear model is closest, and all three models overestimate house 6 (actual \$116,600) by a wide margin. Across all 4,128 test houses, the average absolute difference between predicted and actual price was about \$35,000 for Random Forest, \$39,800 for Gradient Boosting, and \$50,100 for Linear Regression.

Interpretation

The results are consistent with the nature of the models. Linear Regression can only

represent straight-line relationships, so it cannot capture how income, location, and house age interact, and its predictions scatter widely around the actual prices in Figure 5. Random Forest and Gradient Boosting build many decision trees that split the data on these features, so their predictions follow the actual prices more tightly. In this run Random Forest performed slightly better than Gradient Boosting, but this depends on the chosen settings (30 trees of depth 12 versus 160 trees of depth 3) and should not be read as a general ranking. The vertical column of points at \$500,000 in Figure 5 appears because the dataset caps its highest prices at \$500,001 (965 of the 20,640 districts), so the models cannot learn the true value of the most expensive districts. Also, because each record describes a district rather than an individual house, a model can be far off for a single record, as seen for houses 6 and 9.

Achievement of Objectives

The project met its objectives. It loads and prepares a real housing dataset, implements preprocessing and a reproducible train–test split, trains Linear Regression, Random Forest, and Gradient Boosting models from scratch in pure Python, and shows the actual and predicted prices of each model on unseen houses.

Implications

This analyzer can be used in academic environments to teach how regression models work internally, as a reference for comparing model families, and as a starting point for studies on local or user-supplied property datasets. Its outputs demonstrate technique and are not a substitute for professional property valuation.

6 Mapping POs, PSOs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 2: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied programming, statistics, data structures, and machine-learning concepts to build regression models from scratch.
PO2	Problem Analysis	Defined the house-price prediction problem and analyzed data quality (missing values, categorical features) and prediction differences.
PO3	Design/Development of Solutions	Designed a modular pipeline from data loading and preprocessing to model training and price prediction.
PO4	Conduct Investigations of Complex Problems	Used a controlled 80/20 train-test split and compared three model families on unseen data.
PO5	Modern Tool Usage	Used Python and its standard library (csv, math, random, urllib) to implement and run the models.
PO10	Communication	Communicated technical findings through a structured report, console outputs, and charts.

6.2 Program Specific Outcomes (PSOs) Mapping

Table 3: Mapping of Program Specific Outcomes (PSOs) to Project Relevance

PSO No.	Program Specific Outcome	Relevance
PSO1	Foundations of Computing and Problem Solving	Applied programming, data structures, and algorithms (sorting, recursion, matrix solving, decision trees) to build and run regression models from scratch.
PSO2	Software Development and Data-Driven Solutions	Developed a modular, reusable Python application that loads real data, prepares it, trains three models, and reports predicted prices for unseen houses.
PSO3	Emerging Technologies and Research Orientation	Explored machine-learning techniques (ensemble learning, boosting) and analyzed their behavior and limitations on a real dataset.

6.3 Sustainable Development Goals (SDGs) Mapping

Table 4: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
---------	------	-----------

SDG 9	Industry, Innovation and Infrastructure	Demonstrates a data-driven digital tool that supports analytical innovation.
SDG 11	Sustainable Cities and Communities	Housing analytics can support evidence-based planning when applied to appropriate local data.
SDG 12	Responsible Consumption and Production	Automated analysis reduces repetitive manual processing and encourages reproducible work.
SDG 16	Peace, Justice and Strong Institutions	Transparent results and documented assumptions support accountable analytical decision-making.

7 Conclusion

The House Price Data Analyzer demonstrates a complete Python workflow for predicting housing values. It combines data loading, preprocessing, supervised regression, and a clear comparison of actual and predicted prices in a form that can be reused with other tabular datasets. Every component was built in pure Python without external libraries. On the selected California Housing data, Random Forest gave the closest predictions overall, with an average absolute difference of about \$35,000 on the test houses, while Linear Regression provided an interpretable baseline.

The main limitation is that the dataset represents district-level historical observations and does not include every factor used in real property valuation, such as exact address, property condition, renovation history, financing conditions, or current market trends. In addition, the pure Python implementation is slower than optimized libraries, so the ensemble models use a reduced number of trees. Future work should add cross-validation, hyperparameter tuning, explainability with permutation importance, outlier analysis, a user-upload interface, and geographically or temporally separated validation. Any deployment for real decisions should also check fairness, data freshness, and local regulatory requirements.

Bibliography

Online Resources

- [1] Python Software Foundation – Python 3 Documentation
<https://docs.python.org/3/>
- [2] A. Géron – California Housing dataset (housing.csv) used in this project <https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv>
- [3] Kaggle / StatLib – California Housing dataset documentation

Research article references

- [1] Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
- [2] McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.
- [3] VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media.